

MonoDepthNet: A Comprehensive Study on Monocular Depth Estimation in Images

Aditya Mundhara[†], Gadhiya Ronakkumar Vipulbhai[†], Arnava Srivastava[†], Savalia Preet Atulkumar[†], Rohan Regar[†]

[†]Department of Computer Science and Engineering, Indian Institute of Technology Jodhpur, India

Email: {b22ai057, b22ai052, b22ai009, b22ai036, b22cs072}@iitj.ac.in

Github: <https://github.com/InfernoAsura/DeepCV>

Streamlit deploy link: <https://depthcvtemp.streamlit.app/>

Abstract—Monocular depth estimation is a fundamental task in computer vision, aiming to infer scene depth from a single RGB image. In this project, we explore and evaluate various state-of-the-art approaches to monocular depth estimation, combining both traditional computer vision techniques and deep learning models. Our study involves the use of MiDaS, a robust deep learning model trained on diverse datasets for relative depth prediction; Pix2Pix, a conditional GAN framework adapted for supervised depth estimation; and Semi-Global Matching (SGM), a classic stereo-based method adapted for monocular cues. We compare the qualitative and quantitative performance of these models across diverse scenes to understand their strengths and limitations in depth estimation. Our findings highlight the trade-offs between accuracy, generalization, and computational efficiency, offering valuable insights into selecting appropriate depth estimation techniques for real-world applications such as autonomous navigation, augmented reality, and 3D scene reconstruction.

1. Introduction

Understanding the depth of a scene from a single image is a critical yet inherently ill-posed problem in computer vision. Unlike stereo vision systems that leverage multiple viewpoints to compute depth, monocular depth estimation relies solely on a single RGB image, making it highly challenging due to the loss of geometric information. Despite this, recent advancements in deep learning and computer vision have significantly improved the accuracy and applicability of monocular depth estimation methods.

Monocular depth estimation plays a vital role in numerous real-world applications, including autonomous driving, robotics, augmented and virtual reality, and 3D reconstruction. Accurate depth information from monocular cues can enhance scene understanding, facilitate obstacle avoidance, and enable immersive user experiences in virtual environments.

In this project, we aim to explore and compare various approaches to monocular depth estimation. We employ state-of-the-art deep learning models such as MiDaS, known for its robust performance across diverse environments, and Pix2Pix, a generative adversarial network adapted for supervised depth prediction. In addition, we incorporate Semi-Global Matching (SGM)—a classical method traditionally used in stereo

vision—to investigate its feasibility and effectiveness when adapted to monocular inputs.

Our goal is to analyze the performance, generalization ability, and limitations of these models across different scenes. By conducting both qualitative and quantitative evaluations, we aim to identify the most suitable techniques for practical depth estimation tasks and provide insights into the future development of more efficient and accurate monocular depth estimation systems.

2. Methodology

2.1. Objective

The objective of this project is to develop, evaluate, and compare multiple approaches for **monocular depth estimation** using RGB images. These methods span classical stereo vision algorithms and modern deep learning architectures, aiming to predict dense and accurate depth maps in indoor environments.

We explore five distinct approaches:

- **Pix2Pix** [1]: A conditional Generative Adversarial Network (cGAN) that formulates depth estimation as an image-to-image translation task. It combines adversarial training with L1 reconstruction loss to produce visually and structurally consistent depth outputs.
- **Semi-Global Matching (SGM)** [2]: A traditional stereo vision algorithm adapted for monocular depth inference via pseudo-stereo setups. SGM computes pixel-wise matching costs and aggregates them along multiple paths to produce disparity maps.
- **DenseDepth**: A supervised regression model using a DenseNet-121 encoder and a Feature Pyramid Network (FPN) decoder. It leverages multi-scale features and skip connections to produce high-resolution depth maps.
- **DepthAnythingCustom**: A custom encoder-decoder architecture inspired by Depth Anything v2, utilizing an EfficientNet-B0 backbone and FPN-like decoder. It incorporates a composite loss function including L1, SSIM, and edge-aware components to enhance depth accuracy and boundary sharpness.

- **MiDaS**: A pretrained transformer-based model that performs scale-invariant depth estimation. MiDaS generalizes across multiple datasets and produces reliable depth maps without fine-tuning, making it ideal for quick inference and zero-shot evaluation.

By benchmarking these approaches on the NYU Depth V2 dataset, we aim to assess their quantitative performance, structural consistency, and practical applicability. This comparative study provides valuable insights into the effectiveness and trade-offs of various depth estimation strategies in monocular settings.

2.2. Depth Estimation using Pix2Pix

Introduction to Pix2Pix for Depth Estimation

pix2pix [1] is a conditional Generative Adversarial Network (cGAN) designed for general-purpose image-to-image translation tasks. Its effectiveness stems from the combination of adversarial training and pixel-wise reconstruction loss. In the context of depth estimation, Pix2Pix is trained to generate a depth map from an RGB input image, treating this as a structured translation problem. The goal is to map spatial patterns in RGB space to corresponding depth values, capturing object boundaries, surface continuity, and relative distances.

Unlike traditional regression models, Pix2Pix leverages a discriminator network that learns to distinguish real depth maps from generated ones. This adversarial component encourages the generator to produce more realistic, high-fidelity outputs that align with the underlying geometric structure of the scene.

Network Architecture

We adopt the network architecture of Pix2Pix as introduced by [1], with necessary modifications for depth estimation. The Pix2Pix framework consists of two key components:

- **Generator**: A U-Net-based encoder-decoder network.
- **Discriminator**: A PatchGAN-based classifier.

a) Generator Architecture: U-Net with Skip Connections

The generator network is designed to transform high-resolution RGB images into high-resolution depth maps. Given that both input and output are spatially aligned (they represent the same scene), the architecture must preserve fine-grained structural details. To achieve this, we use a **U-Net** [3] architecture, which includes:

- **Encoder**: A series of convolutional layers that progressively downsample the input, capturing hierarchical features and abstract representations of the input image.
- **Decoder**: A set of upsampling layers that reconstruct the depth map from the compressed features.
- **Skip Connections**: Each encoder layer is connected to its corresponding decoder layer via skip connections. These connections concatenate feature maps from earlier layers with those in later layers, allowing low-level features such as edges and boundaries to directly influence the

output. This circumvents the bottleneck and enables better preservation of spatial details.

The encoder-decoder backbone follows a standard pattern: $\text{Conv} \rightarrow \text{BatchNorm} \rightarrow \text{LeakyReLU}$, while the decoder uses $\text{ConvTranspose} \rightarrow \text{BatchNorm} \rightarrow \text{ReLU}$. A Tanh activation is used at the final output to produce normalized depth values.

b) Discriminator Architecture: PatchGAN

The discriminator in Pix2Pix operates not on the entire image but on **local image patches**, classifying whether each patch is real or fake. This **PatchGAN** [4] discriminator offers two major benefits:

- It models high-frequency structures effectively, ensuring local realism in generated depth maps.
- It reduces computational cost by operating on smaller receptive fields rather than full-sized images.

The discriminator is conditioned on both the input RGB image and the corresponding real or generated depth image. It outputs a matrix of probabilities, one for each patch, indicating whether that patch is real or fake.

Markovian Discriminator (PatchGAN)

A critical component of the *pix2pix* [1] framework is the **PatchGAN** discriminator [4], designed to capture **high-frequency structure** in images. Traditional loss functions such as L1 or L2 often produce **blurry** outputs, as they optimize for pixel-wise similarity and fail to account for fine-grained textural details. While L1 loss effectively enforces global or low-frequency accuracy (e.g., shapes, contours), it is not sufficient for generating **realistic surface details**.

To overcome this limitation, the discriminator in *pix2pix* [1] focuses on **local image patches** rather than the full image, hence the name **Markovian discriminator**. The underlying assumption is that textures can be realistically modeled at the patch level, assuming **statistical independence** between patches that are farther apart than the patch size.

The **PatchGAN** discriminator operates as follows:

- It slides a small $N \times N$ receptive field across the input image.
- For each patch, it predicts whether the region is **real** (from the ground truth) or **fake** (generated by the network).
- The final output is the **average** of these patch-level predictions, forming a global **probability map**.

This localized modeling approach significantly reduces the number of parameters, enables faster computation, and allows the network to scale to **arbitrary image sizes**. Moreover, by emphasizing texture and style consistency at the local level, the PatchGAN [4] behaves as an implicit **texture/style loss**, as observed in prior works on texture synthesis and style transfer.

Thus, the PatchGAN discriminator complements the L1 loss as follows:

- **L1 Loss** enforces **global structure** and **low-frequency accuracy**.

- **PatchGAN**[4] enforces **local texture** and **high-frequency sharpness**.

Optimization and Inference

The training of the *pix2pix* [1] network follows the standard **alternating minimax strategy** used in Generative Adversarial Networks (GANs). The optimization process alternates between updating the discriminator and the generator:

- 1) **Update the Discriminator (D)**: One gradient descent step is applied to improve its ability to distinguish between real and generated image pairs. Following [?], the discriminator loss is divided by 2 to slow down its learning, allowing the generator more opportunity to improve.
- 2) **Update the Generator (G)**: Instead of minimizing the original loss $\log(1 - D(x, G(x, z)))$, which can lead to vanishing gradients, the generator is trained to maximize $\log D(x, G(x, z))$. This alternative objective helps the generator produce outputs that better fool the discriminator.

For optimization, we use **minibatch stochastic gradient descent** with the **Adam** [5] optimizer. The parameters are set as follows:

- Learning rate: 0.0002
- $\beta_1 = 0.5$
- $\beta_2 = 0.999$

This configuration has been empirically found to provide stable and effective training dynamics.

Inference Protocol

During inference, only the **generator** is used to convert RGB inputs to their corresponding depth maps. Unlike standard models where dropout [6] and batch normalization are disabled at test time, *pix2pix* [1] continues to use them for better generalization and visual consistency:

- **Dropout**: Applied during inference to introduce randomness and enhance generalization.
- **Instance Normalization**: Batch normalization is applied using the statistics of the current test instance (i.e., batch size = 1), a technique known as **instance normalization**. This has been shown to improve perceptual quality in image generation tasks.

Throughout training and evaluation, the batch sizes are kept small (typically 1) due to GPU memory constraints and dataset size.

Training Objective

The overall training loss for Pix2Pix combines two components:

- 1) **Adversarial Loss (GAN Loss)**: Encourages the generator to produce outputs that the discriminator cannot distinguish from real ones.
- 2) **L1 Loss**: Ensures the generated depth map is close to the ground truth in pixel space, preserving global structure.

The total objective is defined as:

$$\mathcal{L}_{\text{pix2pix}[1]} = \mathcal{L}_{\text{cGAN}}(G, D) + \lambda \mathcal{L}_{L1}(G)$$

Where:

- $\mathcal{L}_{\text{cGAN}}$ is the adversarial loss between generator G and discriminator D ,
- $\mathcal{L}_{L1}$ is the mean absolute error between generated and real depth maps,
- λ controls the trade-off between realism and accuracy (typically set to 100).

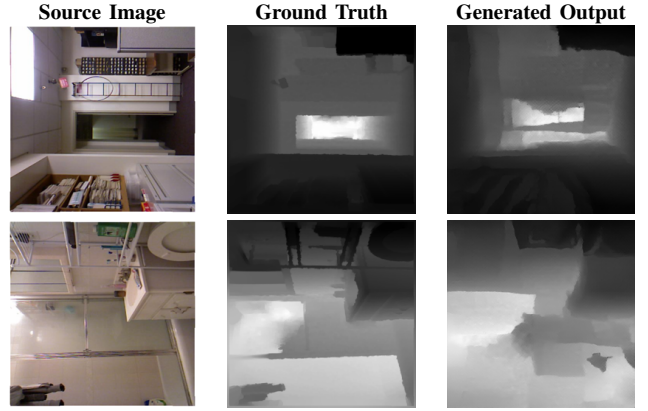


TABLE I
QUALITATIVE COMPARISON BETWEEN SOURCE IMAGES, GROUND TRUTH DEPTH MAPS, AND THE GENERATED OUTPUTS FROM THE PIX2PIX MODEL ON NYU DEPTH DATASET V2 [2].

2.3. Depth Estimation using Semi-Global Matching (SGM)

Semi-Global Matching (SGM) [2] is a robust, cost-efficient algorithm designed to generate high-quality disparity maps from stereo image pairs. Unlike deep learning approaches that formulate depth estimation as a learning problem, SGM uses carefully engineered cost functions combined with dynamic programming-based cost aggregation to approximate a global energy minimization. The technique achieves accurate pixel-level depth estimation by balancing local matching costs with smoothness constraints across multiple image paths.

SGM's effectiveness lies in its ability to handle challenging imaging conditions—such as varying illumination or texture-less regions—using two main ingredients:

- **Robust Matching Cost**: Computed via techniques like the Census Transform (or Mutual Information) and measured using the Hamming distance.
- **Directional Cost Aggregation**: A dynamic programming approach that aggregates costs along multiple 1D paths to impose smoothness in the final disparity map.

2.3.1. Architecture of the Semi-Global Matching (SGM) Model

We adopt the Semi-Global Matching (SGM) architecture as introduced by Hirschmüller, with modifications to improve

robustness and computational efficiency for depth estimation. The SGM framework consists of several key components:

- **Matching Cost Computation:** Measures pixel similarity across stereo image pairs.
- **Cost Aggregation:** Uses multi-directional dynamic programming to enforce smoothness.
- **Disparity Selection:** Applies a winner-takes-all strategy to determine the final depth value for each pixel.
- **Post-Processing:** Enhances accuracy through disparity refinement techniques.

2.3.2. Matching Cost Computation

The first stage of the SGM pipeline involves computing the matching cost for every pixel over a range of candidate disparities. This is crucial for establishing correspondence between the left and right images in a stereo pair.

The cost computation step includes:

- **Mutual Information (MI)-Based Matching:** Unlike simple intensity difference-based approaches, MI is used to handle radiometric variations between images.
- **Census Transform:** A local descriptor that encodes pixel relationships within a window, making the model robust to brightness and contrast changes.
- **Hamming Distance:** The bitwise Hamming distance between the Census-transformed left and right image patches determines similarity.
- **Cost Volume Formation:** The computed pixelwise costs are stored in a 3D cost volume of size (*Height* × *Width* × *Maximum Disparity*), which serves as the input for subsequent processing stages.

2.3.3. Cost Aggregation via Multi-Path Dynamic Programming

The core innovation of SGM lies in its **cost aggregation strategy**, which approximates global optimization by enforcing smoothness constraints along multiple 1D paths.

SGM aggregates cost along **multiple directions**, ensuring that smooth disparity transitions are enforced without compromising depth discontinuities at object boundaries. The directions typically used include:

- **Horizontal:** Left-to-right and right-to-left.
- **Vertical:** Top-to-bottom and bottom-to-top.
- **Diagonal:** Top-left to bottom-right, top-right to bottom-left, and their reverse directions.

The aggregation process follows a dynamic programming update rule:

$$L_r(p, d) = C(p, d) + \min \left(L_r(p - r, d), \right. \quad (1)$$

$$L_r(p - r, d \pm 1) + P1, \quad (2)$$

$$\left. \min_k L_r(p - r, k) + P2 \right) - \min_k L_r(p - r, k) \quad (3)$$

2.3.4. Disparity Selection (Winner-Takes-All Strategy)

Once cost aggregation is complete, the best disparity for each pixel is selected using a **Winner-Takes-All (WTA)** strategy:

$$D(p) = \arg \min_d \sum_r L_r(p, d) \quad (4)$$

- This ensures that the disparity with the **lowest aggregated cost** is chosen for each pixel.
- WTA selection ensures a balance between **pixelwise accuracy and global consistency**.

2.3.5. Disparity Refinement and Post-Processing

To enhance the accuracy of the estimated depth map, the model applies several post-processing techniques:

- **Left-Right Consistency Check:** Removes incorrect matches by ensuring that the left and right disparity maps are mutually consistent.
- **Sub-Pixel Disparity Estimation:** Uses **quadratic interpolation** around the minimum aggregated cost to achieve **sub-pixel accuracy**.
- **Occlusion Handling:** Disparities failing the consistency check are interpolated using surrounding valid pixels.
- **Median Filtering:** A median filter is applied to remove noise while preserving object boundaries.

2.3.6. Multi-Baseline Matching (Optional for Multi-View Stereo)

For applications involving multiple cameras or stereo pairs, SGM extends its approach to **multi-baseline** setups:

- Each stereo pair is processed independently.
- The resulting disparity maps are **fused** using a **weighted mean approach**, normalizing disparities based on their respective baseline distances.

Where:

- $L_r(p, d)$ is the aggregated cost at pixel p and disparity d along direction r .
- $C(p, d)$ is the initial matching cost.
- $P1$ is a penalty for small disparity changes (to encourage smooth surfaces).
- $P2$ is a higher penalty for large disparity jumps (to allow for object edges).

Benefits of Multi-Directional Cost Aggregation:

- **Prevents streaking artifacts** seen in standard dynamic programming methods.
- **Balances smoothness and edge preservation**, ensuring fine details are retained.

Training Objective and Other Sections of the Model

2.3.7. Training Objective

The model aims to optimize a specific loss function that balances reconstruction accuracy and regularization constraints.

The primary objective function consists of the following components:

- 1) **Reconstruction Loss** ($\mathcal{L}_{rec}$): This term ensures that the model can accurately reconstruct input samples by minimizing the difference between input and output. It is typically formulated as Mean Squared Error (MSE) or Cross-Entropy Loss depending on the task.
- 2) **Regularization Term** ($\mathcal{L}_{reg}$): A regularization constraint is introduced to prevent overfitting and enforce smoothness in the learned representations. This can include weight decay, sparsity constraints, or adversarial robustness terms.

The final loss function is a weighted sum of these components:

$$\mathcal{L} = \lambda_1 \mathcal{L}_{rec} + \lambda_2 \mathcal{L}_{reg} \quad (5)$$

where λ_1 and λ_2 are hyperparameters controlling the importance of each term.

2.3.8. Optimization Strategy

The model is trained using Stochastic Gradient Descent (SGD) or Adam optimizer [5], with the following strategies:

- **Mini-batch training:** The dataset is divided into batches to improve convergence.
- **Learning rate scheduling:** Adaptive learning rates (e.g., cosine annealing or step decay) are used to enhance stability.
- **Gradient Clipping:** To prevent exploding gradients, a threshold is applied to gradients.

2.3.9. Regularization Techniques

To improve generalization, the following regularization methods are employed:

- **Dropout:** Randomly drops neurons during training to reduce overfitting.
- **Batch Normalization:** Normalizes feature activations to stabilize training.
- **L2 Weight Decay:** Applies a penalty on large weights to encourage simpler models.

2.3.10. Evaluation Metrics

The model is evaluated using a set of performance metrics specific to the task. Common choices include:

This structured methodology ensures that the model is both accurate and generalizable, optimizing its learning while maintaining robustness.

2.4. Monocular Depth Estimation using DenseDepth

2.4.1. Introduction

Monocular depth estimation is a fundamental problem in computer vision, aiming to predict a depth map from a single RGB image. Unlike stereo-based methods such as Semi-Global Matching (SGM), monocular depth estimation relies



Fig. 1. Depth Map From left and right images, and camera calibration parameters.

purely on learned representations from a single image, making it particularly useful in scenarios where stereo information is unavailable. In this work, we implement and train the **DenseDepth** model, which employs a **DenseNet-121** backbone within a **Feature Pyramid Network (FPN)** to perform depth regression.

2.4.2. Network Architecture

The DenseDepth model is based on a fully convolutional encoder-decoder architecture. The model leverages a **DenseNet-121** encoder to extract hierarchical features from the input image, followed by an FPN decoder to progressively refine the depth map.

Encoder: DenseNet-121

The encoder is based on DenseNet-121, a densely connected convolutional network pretrained on ImageNet. The key characteristics of the encoder are:

- **Dense Connectivity:** Each layer receives feature maps from all preceding layers, improving gradient flow and feature reuse.
- **Pretrained Weights:** The model is initialized with ImageNet-pretrained weights to accelerate convergence.
- **Multi-scale Feature Extraction:** Lower layers capture fine-grained details, while deeper layers encode high-level semantic features.

Decoder: Feature Pyramid Network (FPN)

The decoder utilizes a Feature Pyramid Network (FPN) to upsample and refine depth predictions. The advantages of using FPN include:

- **Multi-scale Feature Fusion:** FPN combines feature maps from different levels of the encoder to improve depth prediction at multiple scales.

- **Skip Connections:** Connections between encoder and decoder layers preserve spatial information lost during downsampling.
- **Progressive Upsampling:** The decoder progressively refines depth maps using transposed convolutions and feature concatenation.

The final output of the model is a single-channel depth map, where each pixel represents the predicted depth value.

2.4.3. Dataset and Preprocessing

We train our model on the **NYU Depth V2** dataset, a widely used benchmark for indoor depth estimation. The dataset consists of:

- 1449 RGB-depth image pairs captured using a Kinect sensor.
- A variety of indoor environments, including bedrooms, offices, and classrooms.
- Depth maps with a resolution of 640×480 pixels, aligned with corresponding RGB images.

Data Augmentation

To improve generalization, we apply the following transformations using **Albumentations**:

- **Geometric Transformations:** Horizontal flips, random resized cropping, affine transformations.
- **Color Jittering:** Adjustments in brightness, contrast, hue, and saturation.
- **Noise Injection:** Gaussian noise, motion blur, and median blur to simulate real-world conditions.

Normalization and Resizing

All input images are resized to 224×224 pixels and normalized using the standard ImageNet mean and standard deviation.

2.4.4. Training Setup

Loss Function

We use Mean Squared Error (MSE) loss to measure the difference between predicted and ground-truth depth maps:

$$\mathcal{L}_{MSE} = \frac{1}{N} \sum_{i=1}^N (D_{pred}^{(i)} - D_{gt}^{(i)})^2 \quad (6)$$

where D_{pred} and D_{gt} represent the predicted and ground-truth depth values, respectively.

Optimization Strategy

The model is trained using the AdamW optimizer with weight decay regularization. The training setup includes:

- **Learning Rate:** 1×10^{-3} , reduced gradually using a OneCycleLR scheduler.
- **Batch Size:** 64 for training, 64 for validation.
- **Gradient Scaling:** Mixed-precision training using PyTorch's autocast and GradScaler for efficiency.

- **Gradient Clipping:** Applied with a max norm of 2.0 to prevent exploding gradients.

The encoder is initially frozen for the first two epochs to allow the decoder to learn meaningful depth features before enabling full fine-tuning.



Fig. 2. Qualitative depth predictions from DenseDepth. From left to right: input image, predicted depth map, ground truth depth map.

2.4.5. Comparison with Other Methods

Compared to classical stereo-based methods such as SGM, DenseDepth has several advantages:

- **Single Image Input:** Unlike SGM, which requires stereo pairs, DenseDepth operates with a single RGB image.
- **Smooth Depth Predictions:** The model learns global scene structures, avoiding the noisy disparity maps produced by traditional methods.
- **Computational Efficiency:** Once trained, inference is fast and does not require costly disparity aggregation.

2.4.6. Conclusion

We successfully implemented and trained the **DenseDepth** model using a **DenseNet-121** backbone within an FPN architecture for monocular depth estimation. The model demonstrated strong generalization on the NYU Depth V2 dataset, achieving high SSIM scores while producing visually accurate depth maps. Future work includes experimenting with alternative loss functions, such as Scale-Invariant Loss, and integrating attention mechanisms to enhance depth estimation.

2.5. Monocular Depth Estimation using DepthAnything

2.5.1. Introduction

Monocular depth estimation aims to predict a dense depth map from a single RGB image. In contrast to stereo-based methods, monocular approaches learn to infer scene geometry from a single view by leveraging powerful feature extractors and an effective decoder. In our work, we implement a model named **DepthAnythingCustom**, which is inspired by the Depth Anything v2 approach. The model employs a pretrained **EfficientNet-B0** encoder and a custom FPN-like decoder with skip connections. This architecture is designed to extract multi-scale features and fuse them to produce accurate depth predictions, making it suitable for indoor scenes such as those provided by the NYU Depth V2 dataset.

2.5.2. Network Architecture

Encoder: EfficientNet-B0

The encoder leverages a pretrained **EfficientNet-B0** model using the `timm` library. Key characteristics of the encoder include:

- **Pretrained Features:** The network is initialized with ImageNet-pretrained weights, enabling robust feature extraction from RGB images.
- **Multi-scale Feature Extraction:** The encoder outputs features from several layers. These hierarchical features capture both low-level textures and high-level semantic information.
- **Feature Extraction Utility:** The encoder is created with `features_only=True`, returning a list of feature maps from various resolution stages.

Decoder: FPN-like with Skip Connections

The decoder is built in a top-down manner, combining features from different scales via lateral connections:

- **Lateral Connections:** For each encoder feature (except the deepest layer), a 1×1 convolution is applied to reduce the channel dimensions (e.g., to 256 channels).
- **Top-Down Pathway:** Starting from the deepest encoder feature, the feature map is progressively upsampled using bilinear interpolation. At each stage, the upsampled feature is processed through a 3×3 convolution followed by batch normalization and ReLU activation.
- **Feature Fusion:** The upsampled features are added to the corresponding lateral connection outputs, effectively merging context from different scales.
- **Depth Head:** The final feature map is passed through a sequence of convolutional layers, reducing the number of channels step-by-step, before outputting a single-channel depth map using a 1×1 convolution.

Model Flow

- 1) The input RGB image ($3 \times 224 \times 224$) is passed through the **EfficientNet-B0** encoder to extract multi-scale features.
- 2) The decoder follows a top-down pathway: For each scale, lateral features are combined with upsampled features from the previous (deeper) layer.
- 3) The final output from the decoder is fed into the depth head, producing an initial depth prediction.
- 4) If necessary, the predicted depth map is resized via bilinear interpolation to match the input dimensions.

2.5.3. Dataset and Preprocessing

The model is trained on the **NYU Depth V2** dataset, which includes:

- RGB and corresponding depth images captured indoors.
- A variety of scenes such as classrooms, offices, and bedrooms.
- Typical preprocessing includes normalization using ImageNet statistics.

Data Augmentation

Data augmentation is implemented via **Albumentations** to improve model robustness:

- **Geometric Augmentations:** Horizontal flips, random resized cropping, and affine transformations (translation, scaling, rotation).
- **Color Augmentations:** Random brightness and contrast adjustments, RGB shifts, and color jitter.
- **Noise Augmentations:** Gaussian noise, motion blur, and median blur to simulate challenging imaging conditions.

The training transformations resize the images to 224×224 and normalize them using standard ImageNet mean and standard deviation.

2.5.4. Training Configuration

Loss Function

The training loss comprises a weighted sum of multiple terms to enforce both overall pixel accuracy and structural consistency:

- **L1 Loss:** Measures pixel-wise differences between predicted depth and ground-truth.
- **SSIM Loss:** Encourages the preservation of structural similarity.
- **Edge Loss:** An edge-aware component, based on Sobel operators, to promote sharper depth boundaries.

The overall loss is defined as:

$$\mathcal{L} = 0.5 \cdot \mathcal{L}_{L1} + 0.3 \cdot (1 - \text{SSIM}) + 0.2 \cdot \mathcal{L}_{edge} \quad (7)$$

Optimization and Training Strategy

- **Optimizer:** AdamW is used with a learning rate typically set to 1×10^{-3} . Only the decoder is initially trained while the encoder is frozen.

- **Learning Rate Scheduler:** A `ReduceLROnPlateau` scheduler adapts the learning rate based on the validation loss.
- **Mixed Precision Training:** Leveraging PyTorch’s `autocast` and `GradScaler` for computational efficiency.
- **Batch Size and Epochs:** Batch size is set (e.g., 8 or 16) with training for 10 epochs.

2.5.5. Evaluation Metrics

The model is evaluated using a combination of standard metrics for depth estimation:

- **Absolute Relative Error, RMSE, RMSE (log):** Quantitative measures of depth prediction error.
- **Thresholded Accuracy (a1, a2, a3):** The percentage of predictions within certain error thresholds.
- **SSIM and other Quality Measures:** To assess the structural similarity between the predicted and ground-truth depth maps.

2.5.6. Experimental Results

Qualitative Results

Figure 3 shows examples of depth predictions, including:

- Input RGB images,
- Ground-truth depth maps,
- Predicted depth maps using a color mapping (e.g., magma colormap) for visualization.
- Error map

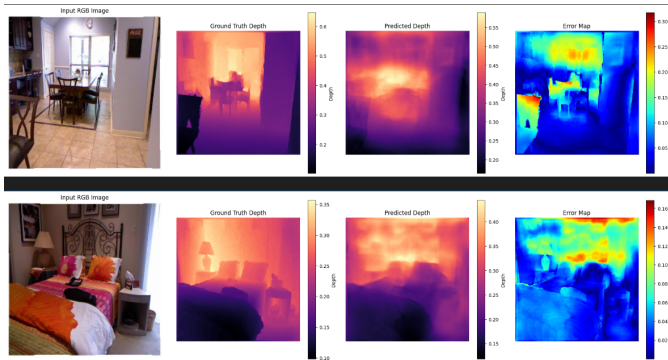


Fig. 3. Qualitative depth predictions from DepthAnythingCustom model. From left to right: Input RGB image, Ground-truth depth, Predicted depth, and Error Map.

2.5.7. Conclusion

The proposed **DepthAnythingCustom** model leverages a pre-trained **EfficientNet-B0** encoder and an FPN-like decoder with skip connections to tackle monocular depth estimation. With comprehensive data augmentation and a composite loss function, the model learns to produce accurate depth maps on the NYU Depth V2 dataset. Future work includes further experimentation with different backbones, loss functions, and

fine-tuning strategies to enhance performance and generalization.

2.6. Depth Estimation using MiDaS

MiDaS (Monocular Depth Estimation via a Scale-Invariant Transformer) is a state-of-the-art model designed to predict relative depth from a single RGB image. It generalizes well across diverse scenes and datasets due to its training on a large and varied collection of depth data.

In this work, we use the lightweight version, `MiDaS_small`, which provides a good trade-off between inference speed and accuracy, making it suitable for real-time applications.

2.6.1. Model Architecture

`MiDaS_small` is a convolutional neural network based on a ResNet-like encoder and a decoder adapted for dense depth prediction. The architecture is trained using a scale-invariant loss to focus on preserving relative depth rather than absolute values, making it robust to variations in scene scale.

The model accepts an RGB image of size 256×256 as input and outputs a relative depth map. For our experiments, images were resized to this resolution using torchvision transforms. After inference, the depth predictions were upsampled to match the original ground truth resolution (240×320) using bilinear interpolation.

2.6.2. Evaluation

The model was evaluated on the NYU Depth V2 dataset using standard depth estimation metrics. Predicted depth maps were aligned with the ground truth via median scaling. Only valid pixels (where ground truth > 0) were considered in the evaluation.

The following metrics were computed:

- **Mean Absolute Error (MAE):** Measures the average of absolute differences between predicted and ground truth depths.
- **Root Mean Squared Error (RMSE):** Penalizes larger errors more heavily, giving insight into prediction consistency.
- **Absolute Relative Error (AbsRel):** Captures the mean of the absolute differences relative to ground truth values.
- **Squared Relative Error (SqRel):** Similar to AbsRel but squares the differences, emphasizing larger deviations.
- **Threshold Accuracy (δ_1):** Percentage of pixels where the ratio between the prediction and ground truth is less than 1.25.

The evaluation was conducted over 10 batches of the dataset (approximately 160 samples). MiDaS demonstrated strong generalization capabilities and competitive performance, particularly in maintaining structural consistency and relative depth gradients.

The average results obtained were:

- **Average RMSE:** 0.2778

- **Average MAE:** 0.2252
- **Average AbsRel:** 0.2441
- **Average SqRel:** 0.0968
- **Threshold Accuracy** ($\delta_1 < 1.25$): 0.7167

These results demonstrate MiDaS’s effectiveness for scale-invariant monocular depth estimation, especially in zero-shot or non-finetuned scenarios.

3. Experiments and Evaluation

3.1. Dataset

To evaluate the performance of our depth estimation model based on the *pix2pix* [1] framework, we utilize the **NYU Depth V2** dataset [2]. This dataset is widely used for benchmarking indoor depth estimation tasks and provides high-quality, aligned **RGB-Depth image pairs** captured using the **Microsoft Kinect sensor**.

The dataset includes:

- **1449 RGB-depth image pairs**, densely labeled and pre-aligned.
- A wide variety of **indoor environments**, including bedrooms, living rooms, classrooms, offices, and more.
- Each pair includes an **RGB image** and the corresponding **ground truth depth map**.

3.2. Qualitative Analysis

A visual inspection of the predicted depth maps in Table II reveals that the model captures the general layout of indoor scenes effectively. The predicted maps maintain sharp boundaries around key structural elements such as walls, furniture, and objects.

The incorporation of **skip connections in the generator** allows for the retention of low-level details, and the use of a **PatchGAN discriminator** encourages the generation of realistic textures and high-frequency details. Together, these architectural choices contribute to the structural coherence and visual plausibility of the depth estimations.

Qualitative results across all models—Pix2Pix, DenseDepth, DepthAnythingCustom, MiDaS, and SGM—demonstrate their effectiveness in capturing the overall depth structure and spatial layout of indoor scenes. Pix2Pix and DenseDepth maintain fine structural details and object boundaries due to their use of skip connections and encoder-decoder architectures. DepthAnythingCustom produces smooth and edge-aware predictions through a composite loss and multi-scale feature fusion. MiDaS excels at generalization and provides scale-invariant depth estimations. Despite being a classical method, SGM yields coherent disparity maps with strong boundary localization. Visual comparisons reveal each model’s unique strengths in preserving textures, ensuring depth consistency, and maintaining structural integrity.

3.3. Evaluation Metrics

To quantitatively assess the quality of the predicted depth maps, we employ the following metrics:

- **Mean Absolute Error (MAE):** Measures the average absolute difference between predicted and ground truth depth values. Lower MAE indicates better performance.
- **Root Mean Squared Error (RMSE):** Provides a penalized average error, giving higher weight to larger errors. It is sensitive to outliers and often used to assess global prediction accuracy.
- **Structural Similarity Index Measure (SSIM)** [7]: Evaluates the perceptual similarity between the predicted and ground truth depth maps by comparing local patterns of pixel intensities. SSIM scores range from 0 to 1, with higher values indicating greater structural fidelity.

3.4. Results

The *pix2pix* [1] model was trained using RGB images as input and the corresponding depth maps as target outputs. After sufficient epochs of training on the NYU Depth V2 dataset [2], the model achieved the evaluation metrics on the test set in Table III

Model	MAE	RMSE	SSIM
Pix2Pix	0.0589	0.0946	0.7345
DenseDepth	0.0550	0.0970	0.8381
DepthAnythingCustom	0.2324	0.0774	0.6498
MiDaS	0.2252	0.2778	0.7167

TABLE III
EVALUATION METRICS (MAE, RMSE, SSIM) FOR DIFFERENT DEEP LEARNING MODELS ON THE NYU DEPTH V2 DATASET.

These results demonstrate that the model is capable of learning meaningful depth representations from the visual cues present in RGB images. The low MAE and RMSE indicate good pixel-wise accuracy, while the relatively high SSIM shows that the model effectively preserves structural information, including object boundaries and depth discontinuities.

4. Conclusion and Limitations

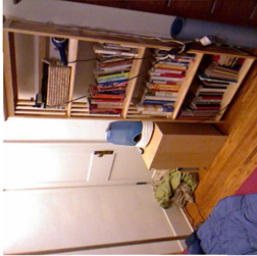
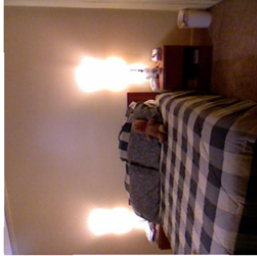
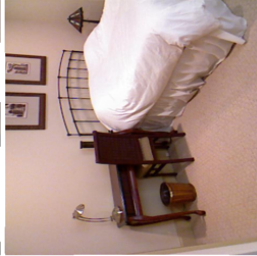
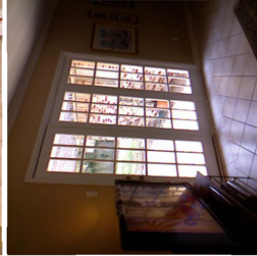
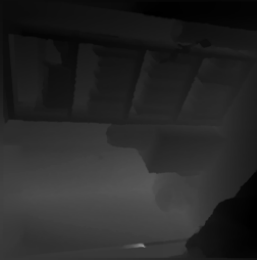
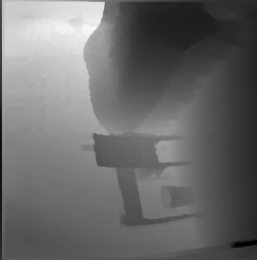
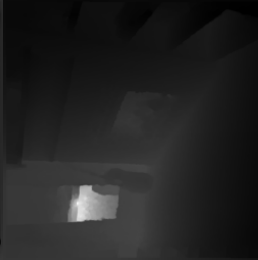
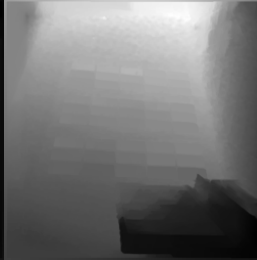
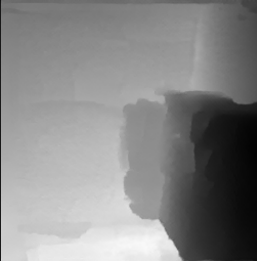
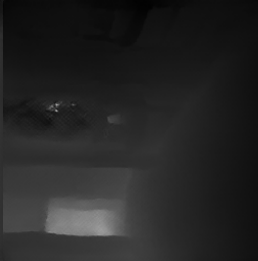
5. Contributions

- **Preet Savalia** researched on *pix2pix* architecture, training on depth estimation, fine-tuning and evaluation, report writing.
- **Ronak Gadhiya** worked on deep learning architecture based model inspired by DepthAnything, also trained DenseDepth, demo app.
- **Aditya Mundhara** worked on DenseDepth (approach using DenseDepth encoder), finetuning and evaluating DepthAnything, demo app.
- **Rohan Regar** worked on tuning MiDaS and SGM (cv based approach), demo app.
- **Arnav Srivastava** worked on Semi-Global matching implementation, comparison with MiDaS, report.

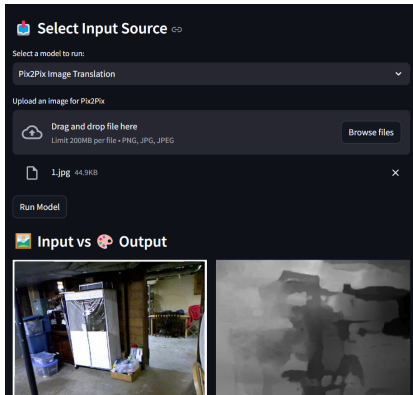
References

- [1] P. Isola, J.-Y. Zhu, T. Zhou, and A. A. Efros, “Image-to-image translation with conditional adversarial networks,” 2018.

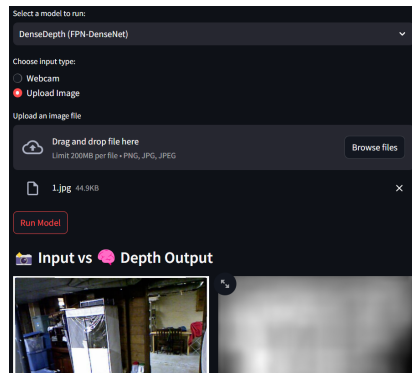
TABLE II
QUALITATIVE RESULTS: SOURCE IMAGES, GROUND TRUTH DEPTHS, AND GENERATED DEPTHS ACROSS DIFFERENT SAMPLES USING PIX2PIX [1].

	Sample 1	Sample 2	Sample 3	Sample 4	Sample 5
Source					
GT Depth					
Generated					

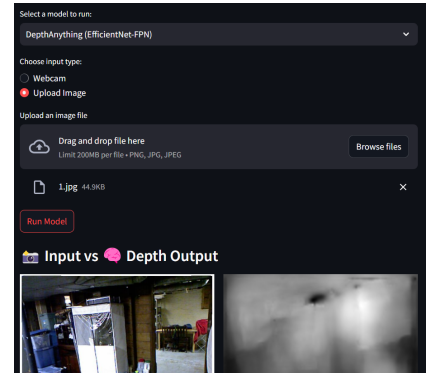
- [2] Hirschmuller, Heiko, "Stereo processing by semiglobal matching and mutual information," 2008.
- [3] O. Ronneberger, P. Fischer, and T. Brox, "U-net: Convolutional networks for biomedical image segmentation," 2015.
- [4] U. Demir and G. Unal, "Patch-based image inpainting with generative adversarial networks," 2018.
- [5] D. P. Kingma and J. Ba, "Adam: A method for stochastic optimization," 2017.
- [6] G. E. Hinton, N. Srivastava, A. Krizhevsky, I. Sutskever, and R. R. Salakhutdinov, "Improving neural networks by preventing co-adaptation of feature detectors," 2012.
- [7] J. Nilsson and T. Akenine-Möller, "Understanding ssim," 2020.



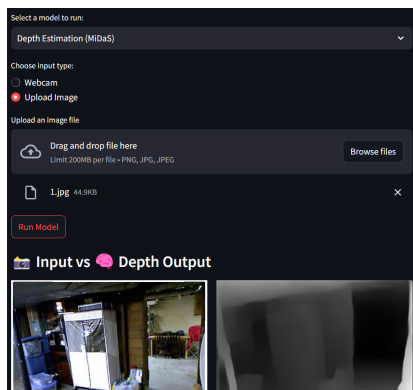
Pix2Pix



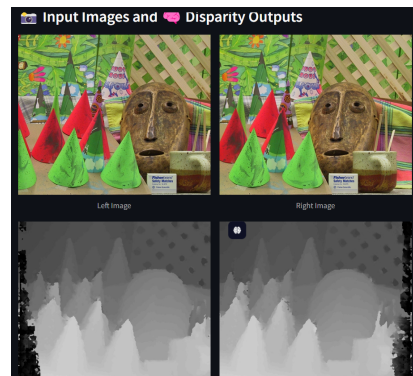
DenseDepth



DepthAnythingCustom



MiDaS



SGM

Fig. 4. Qualitative demo app results for different monocular depth estimation models. Each image shows the predicted depth map for the same input scene using the respective model.